

JoSIM Optimisation Harness

Refactoring and Extension Design Document

Point 1: Architecture Refactor | Point 2: Bayesian Optimiser

1 Motivation and Goals

The existing harness couples four concerns inside the parameter class and main loop:

- * Symbol-table and template-variable management (~param~ substitution into spice deck)
- * Loop-iteration machinery (scan, future binary-search, spiral, ...)
- * Stateful-operator lifecycle (sums, LSQ fits, future min/max/avg/geomean)
- * Complete Simulated Annealing implementation (all state stored as class statics)

The refactor separates these into distinct classes with clean interfaces, then adds Bayesian Optimisation (BO) as a first-class optimiser alongside SA.

Point	Scope
1	Structural refactor: LoopComplex, LoopIterator, StatefulExpression, Optimizer hierarchy, EvalCache
2	BayesianOptimizer using BayesOpt/limbo + NLOpt + Eigen
3	Little-language review: Flex/Bison grammar, new keywords, C++ wrapper cleanup
4	Overall code review: CMake, git, C++ idioms, KDevelop

2 High-Level Architecture

Top-level call structure after refactoring:

```
main()
|
|-- parse spice deck -> builds LoopComplex from pragma definitions
|
|-- plain mode : loop_complex.run_once()
|-- SA mode : SimulatedAnnealing(loop_complex).run()
+-- BO mode : BayesianOptimizer(loop_complex).run()
```

Key invariant: LoopComplex::run_once() is the stable oracle. SA and BO are entirely outside it, treating it as a black box returning a double (or NaN if no eval expression is defined).

3 Class Breakdown

3.1 parameter -- symbol table and template variable

Stripped to its essential role. All SA state and loop-iterator state are removed. Flags use bit-fields consistent with the existing codebase style.

```
enum class ParamScale { Linear, Log };

class parameter {
    char          *name;
    static std::vector<parameter*> all_params; // replaces linked list

    enum class Type { Constant, Assignment, Scan, BinSearch } type;
    double        cur_value;
    expression    *expr;          // Assignment type only, nullptr otherwise

    // Attributes -- bit-fields, consistent with existing style
    unsigned      in_summary : 1; // 0 -> suppressed from summary file
    unsigned      optimizable : 1; // eligible for SA/BO perturbation
    ParamScale    scale;
    double        min_value, max_value;
    unsigned      last_updated;      // = n_josim_runs at last pull

public:
    static parameter *find_parameter(const char *nm); // linear search
    double get_cur_value();
    void set_value(double v);
    double to_normalized(double v) const;
    double to_physical (double u) const;
    static void update_assignments();
    static unsigned list_names (FILE *fp);
    static void list_values(FILE *fp);
};
```

Stale-value detection: last_updated compared to global n_josim_runs. No new state introduced.

Normalisation: Log or linear per parameter. Log is the default for circuit components because behaviour scales multiplicatively. Equal distances in log space correspond to equal ratios.

3.2 expression class hierarchy

The base expression class is stateless -- it provides a current value via recursive pull evaluation and knows nothing about lifecycle or state. Stateful variants are subclasses that add their own accumulator state and self-register with LoopComplex on construction.

```
// Base class: stateless, pure pull evaluation
class expression {
    exp_type    type;
    expression  *l_arg, *r_arg, *t_arg;
```

```

    double      value;
    double      (*func1_ptr)(double x);
    double      (*func2_ptr)(void *obj_ptr, double x);
    void        *obj_ptr;
public:
    // constructors as before
    virtual double get_value(); // recursive, no state knowledge
};

// Interface for stateful subclasses -- registered with LoopComplex
class StatefulExpression {
public:
    unsigned level;           // nesting level at time of registration
    virtual void initialize() = 0; // called before first iteration of level
    virtual void update()     = 0; // called after every JoSIM run
    virtual void finalize()   = 0; // called when level exhausted
};

// Stateful subclasses: inherit both
class sum_expression : public expression, public StatefulExpression {
    double accumulator;
public:
    sum_expression(expression *arg); // registers with LoopComplex
    double get_value() override;     // returns accumulator
    void initialize() override { accumulator = 0.0; }
    void update() override { accumulator += l_arg->get_value(); }
    void finalize() override { } // no-op for sum
};

class avg_expression : public expression, public StatefulExpression {
    double sum;
    unsigned count;
public:
    void initialize() override { sum = 0.0; count = 0; }
    void update() override { sum += l_arg->get_value(); count++; }
    void finalize() override { } // get_value() divides on demand
    double get_value() override { return count ? sum/count : NAN; }
};

class geomean_expression : public expression, public StatefulExpression {
    double log_sum;
    unsigned count;
public:
    void initialize() override { log_sum = 0.0; count = 0; }
    void update() override { log_sum += log(l_arg->get_value()); count++; }
    void finalize() override { }
    double get_value() override { return count ? exp(log_sum/count) : NAN; }
};

```

Note: *min_expression* and *max_expression* follow the same pattern. *LsqFit* is not an expression subclass -- it is a separate object referenced via *func2_ptr/obj_ptr*, registered in a separate *LoopComplex* list so it can always be finalized after *StatefulExpressions* at the same level.

3.3 LoopIterator hierarchy -- iteration strategies

Each iterator owns one parameter and knows how to step it. The log/linear mapping lives in the parameter via `to_physical()`, so the iterator always steps `u` uniformly through `[0,1]` -- a log-scaled scan is just a `ScanIterator` on a log-scaled parameter, no separate `LogScanIterator` needed.

```
class LoopIterator {
    parameter *param;
    unsigned level;          // 0 = innermost
public:
    virtual void initialize() = 0;
    virtual bool next()      = 0;  // false when exhausted
    virtual ~LoopIterator() = default;
};

class ScanIterator : public LoopIterator {
    double v_start, v_end;
    unsigned n_steps, step_cntr;

    void initialize() override {
        step_cntr = 0;
        param->set_value(param->to_physical(0.0));
    }
    bool next() override {
        if (++step_cntr >= n_steps) return false;
        double u = (double)step_cntr / (n_steps - 1);
        param->set_value(param->to_physical(u));
        return true;
    }
};

class BinarySearchIterator : public LoopIterator {
    expression *probe;      // expression whose sign change is sought
    double tolerance;       // in normalised [0,1] space
    double lo, hi;

    void initialize() override {
        lo = 0.0; hi = 1.0;
        param->set_value(param->to_physical(0.5));
    }
    bool next() override {
        if ((hi - lo) < tolerance) return false; // converged
        double mid = 0.5 * (lo + hi);
        if (probe->get_value() < 0.0) lo = mid;
        else hi = mid;
        param->set_value(param->to_physical(0.5*(lo+hi)));
        return true;
    }
};
```

3.4 LoopComplex -- the evaluation oracle

LoopComplex is built entirely by the parser from pragma definitions. It holds two separate registries for stateful objects, ensuring the correct finalization order at each level: StatefulExpressions always before LsqFits.

```
class LoopComplex {
    std::vector<LoopIterator*>    iterators;    // in registration order
    std::vector<StatefulExpression*> stateful;    // expressions: sum, avg, ...
    std::vector<lsq_fit_function*> lsq_fits;    // finalized after stateful
    parameter                    *eval_expr;    // nullptr -> run_once() returns NaN
    unsigned                     current_level;

public:
    // Called by constructors of stateful subclasses and lsq_fit_function:
    void register_stateful(StatefulExpression *s) {
        s->level = current_level;
        stateful.push_back(s);
    }
    void register_lsq_fit(lsq_fit_function *f) {
        f->level = current_level;
        lsq_fits.push_back(f);
    }
    // Called by LoopIterator constructors:
    void register_iterator(LoopIterator *it) {
        it->level = current_level++; // post-increment: iterator owns this level
        iterators.push_back(it);
    }
    unsigned current_nesting_level() const { return current_level; }

    double run_once();
};
```

run_once() pseudo-code

```
double LoopComplex::run_once() {
    // 1. Initialise all levels outermost -> innermost
    for (auto *it : iterators) it->initialize();
    for (auto *s : stateful) s->initialize();
    for (auto *f : lsq_fits) f->initialize();

    // 2. Inner simulation loop
    for (;;) {
        parameter::update_assignments();
        circuit.write_cir_file();
        run_josim();
        parse_josim_output();
        for (auto *s : stateful) s->update();
        for (auto *f : lsq_fits) f->add_data();

        // 3. Advance levels innermost -> outermost
        bool advanced = false;
        for (unsigned lv = 0; lv < iterators.size(); lv++) {
```

```

        if (iterators[lv]->next()) { advanced = true; break; }

        // level lv exhausted -- finalize in correct order
        for (auto *s : stateful)
            if (s->level == lv) s->finalize();
        for (auto *f : lsq_fits)
            if (f->level == lv) f->finalize();

        if (lv == iterators.size() - 1) {
            // outermost level done
            return eval_expr ? eval_expr->get_cur_value() : NAN;
        }

        // restart this level
        iterators[lv]->initialize();
        for (auto *s : stateful)
            if (s->level == lv) s->initialize();
        for (auto *f : lsq_fits)
            if (f->level == lv) f->initialize();
    }
    if (!advanced) break; // no iterators -- single pass
}
return eval_expr ? eval_expr->get_cur_value() : NAN;
}

```

Level convention: level 0 is the innermost loop, consistent with the existing parameter.h comment. The first-registered iterator is the outermost (highest level number), the last-registered is innermost (level 0). StatefulExpressions and LsqFits registered between two scan pragmas get the level of the inner one.

Finalization order: StatefulExpressions always finalize before LsqFits at each level. This is structural -- the two separate registries ensure it without any priority field.

3.5 Optimizer hierarchy

```

class Optimizer {
protected:
    LoopComplex &oracle;
    EvalCache    cache;
public:
    Optimizer(LoopComplex &lc) : oracle(lc), cache(lc.n_optimizable_params()) {}
    virtual void run() = 0;
    virtual ~Optimizer() = default;
};

class SimulatedAnnealing : public Optimizer {
    double          temperature, best_score;
    std::vector<an_sched> schedule;
    FILE            *log_file;
    int (*reject_fn)() = nullptr; // optional, internal to SA only
    bool find_valid_change();
    void run() override;
};

```

```
};

class BayesianOptimizer : public Optimizer {
    GaussianProcess      gp;
    ObservationHistory    history;
    AcquisitionFunction    acquisition;
    void run() override;
};
```

3.5.1 SA reject mechanism

The reject function pointer is internal to SimulatedAnnealing. It provides a cheap binary pre-check before committing to a full oracle evaluation. The two-level retry loop mirrors the original anneal.cpp design.

```
bool SimulatedAnnealing::find_valid_change() {
    for (each optimizable parameter, random order) {
        for (int attempt = 0; attempt < MAX_CHANGE_TRIES; attempt++) {
            change_param();
            if (reject_fn == nullptr || reject_fn() == 0)
                return true;
            unchange_param();
        }
    }
    return false;
}
```

Reject vs negative eval: reject fires -> invalid config, retry without consuming an SA step. Negative eval -> valid but failing circuit, goes through acceptance probability. Negative scores mark the feasibility boundary and are valuable BO observations.

4 EvalCache

Advisory within-run deduplication cache. Owned by the Optimizer instance. Not persisted across runs. The entry layout uses manual struct sizing to avoid wasting memory on a fixed MAX_PARAMS array -- the exact parameter count is known at construction time.

Entry layout

```
struct Entry {
    uint64_t hash;    // 0 = empty slot; nonzero = occupied (sentinel trick)
    double    score;  // fixed location at offset 8, always accessible
    // n_params doubles follow immediately in memory:
    double *params() {
        return reinterpret_cast<double*>(this + 1);
    }
    // uint64_t and double are both 8 bytes -> no padding, natural alignment
};
```

```
// Entry size at runtime:
//   sizeof(Entry) + n_params * sizeof(double)
//   = 16 + n_params * 8   bytes
```

Cache class

```
class EvalCache {
    static constexpr double EPS      = 1e-6;
    static constexpr double EPS_SQ   = EPS * EPS;
    static constexpr int    MAX_PROBE = 3;

    char    *table;          // flat block: capacity * entry_sz bytes
    size_t   capacity;       // power-of-2, fixed at construction
    size_t   n_params;       // set at construction
    size_t   entry_sz;       // = sizeof(Entry) + n_params * sizeof(double)
    size_t   n_entries;
    size_t   n_hits;

    Entry *get_entry(size_t i) const {
        return reinterpret_cast<Entry*>(table + i * entry_sz);
    }

    uint64_t compute_hash(const double *p) const;
    // hash == 0 -> bump to 1 to preserve sentinel

public:
    EvalCache(size_t n_params, size_t capacity = 4096);
    ~EvalCache() { delete[] table; }
    EvalCache(const EvalCache&) = delete;
    EvalCache& operator=(const EvalCache&) = delete;

    std::optional<double> lookup(const double *p);
    void store (const double *p, double score);
    size_t size() const { return n_entries; }
    size_t hits() const { return n_hits; }
};

EvalCache::EvalCache(size_t n_params, size_t capacity)
    : capacity(capacity), n_params(n_params)
    , entry_sz(sizeof(Entry) + n_params * sizeof(double))
    , n_entries(0), n_hits(0)
{ table = new char[capacity * entry_sz](); // zero-init -> all hashes = 0 = empty
}
```

Lookup

```
std::optional<double> EvalCache::lookup(const double *p) {
    uint64_t h = compute_hash(p);
    if (h == 0) h = 1; // preserve sentinel
    size_t idx = h & (capacity - 1);
```



```

    for (int probe = 0; probe < MAX_PROBE; probe++) {
        size_t i = (idx + probe*probe) & (capacity - 1);
        Entry *e = get_entry(i);
        if (e->hash == 0) return std::nullopt; // empty slot
        if (e->hash == h) {
            // cheap hash match: now verify with Euclidean norm
            const double *ep = e->params();
            double dist_sq = 0.0;
            for (size_t k = 0; k < n_params; k++) {
                double d = ep[k] - p[k];
                dist_sq += d*d;
            }
            if (dist_sq < EPS_SQ) { n_hits++; return e->score; }
        }
    }
    return std::nullopt;
}

```

Design decision	Rationale
hash == 0 sentinel	Eliminates separate occupied flag; entire table zero-init = all empty
score at fixed offset 8	Always accessible without index arithmetic once hash matches
Variable-length params via pointer wrangling	No wasted memory from MAX_PARAMS; exact size known at construction
uint64_t hash + double params	Both 8 bytes; no padding between header and params array
Power-of-2 capacity	Cheap & masking for index; new char[n]() zero-initialises
No rehash	Cache is advisory and transient; size generously at construction
No persistence	SA runs many hours; invalidation complexity far exceeds benefit

5 Parameter Normalisation

All optimiser arithmetic in normalised [0,1] space. Physical values only used for ~param~ substitution into the spice deck.

Parameter type	Mapping
Inductances, capacitances, resistances	Log
Critical currents, bias currents > 0	Log
Bias currents passing through zero	Linear (log undefined at zero)
Phase values, angles	Linear
Constrained Cartesian coordinates	Linear (reject function enforces constraint)

```
// Log:    to_normalized(v) = (log(v) - log(vmin)) / (log(vmax) - log(vmin))
//         to_physical(u)   = vmin * pow(vmax/vmin, u)

// Linear: to_normalized(v) = (v - vmin) / (vmax - vmin)
//         to_physical(u)   = vmin + u*(vmax - vmin)
```

Note: Log space: equal distances = equal ratios, matching circuit physics. GP metric reflects physical similarity. Ellipsoid axes in log space report directly as +/-% tolerance per parameter.

6 Little Language: Parameter Declaration

Each parameter declaration is a single complete statement that defines both the parameter and its role. There is no separate characterisation step. The keyword after .opt determines what entity is created atomically.

6.1 Statement forms

```
* *Pragma param   Ic1   value=250u  min=200u  max=300u  scale=log  optimize
* *Pragma param   Rshunt assign=(1.0/Ic1 * k)  nosummary
* *Pragma scan    Ic2   start=180u  stop=220u  steps=20  scale=log
* *Pragma bsearch Vbias min=1.0  max=2.0  probe=some_expression  tolerance=1e-3
* *Pragma eval    (expression)
* *Pragma reject  (expression)
```

Keyword	Entity created
param + value=	Constant parameter, no iterator
param + assign=	Assignment parameter (expression), no iterator
scan	Parameter + ScanIterator registered with LoopComplex
bsearch	Parameter + BinarySearchIterator registered with LoopComplex
eval	Sets LoopComplex eval_expr (required for SA/BO, optional for sweep)
reject	Sets SA reject function expression (optional)

6.2 Parse-time validation

Complete specification enables immediate validation at parse time with meaningful yylineno error messages:

- * optimize requires min and max to be present
- * assign= and value= are mutually exclusive
- * scale=log requires min > 0
- * scan requires start, stop, steps
- * bsearch requires min, max, probe, tolerance

- * Parameters referenced in assign= or probe= must already be defined (statement order enforced)

6.3 Lexer changes (Point 3 scope)

The existing lexer already handles = as ASSIGN token within the IN_PRAGMA state, and already separates pragma context from expression context via the IN_PRAGMA exclusive state. New keyword tokens must be added before the general SYMBOL rule to take precedence:

```
// New tokens to add in IN_PRAGMA state (before general SYMBOL rule):
<IN_PRAGMA>"value"      { return KW_VALUE; }
<IN_PRAGMA>"min"        { return KW_MIN; }
<IN_PRAGMA>"max"        { return KW_MAX; }
<IN_PRAGMA>"scale"      { return KW_SCALE; }
<IN_PRAGMA>"log"        { return KW_LOG; }
<IN_PRAGMA>"linear"     { return KW_LINEAR; }
<IN_PRAGMA>"optimize"   { return KW_OPTIMIZE; }
<IN_PRAGMA>"nosummary"  { return KW_NOSUMMARY; }
<IN_PRAGMA>"steps"      { return KW_STEPS; }
<IN_PRAGMA>"start"      { return KW_START; }
<IN_PRAGMA>"stop"       { return KW_STOP; }
<IN_PRAGMA>"probe"      { return KW_PROBE; }
<IN_PRAGMA>"tolerance"  { return KW_TOLERANCE; }
<IN_PRAGMA>"assign"     { return KW_ASSIGN; }
<IN_PRAGMA>"eval"       { return KW_EVAL; }
<IN_PRAGMA>"reject"     { return KW_REJECT; }
<IN_PRAGMA>"bsearch"    { return KW_BSEARCH; }
```

Note: Renaming the ASSIGN token to EQUALS (Point 3 cleanup) would improve grammar readability: EQUALS unambiguously means the = character, whereas ASSIGN currently does double duty as both keyword and punctuation.

6.4 assign= expression delimiter

The parentheses in assign=(expr) and probe=(expr) serve as natural delimiters, preventing the expression parser from consuming subsequent keyword tokens as part of the expression. Parentheses are required for these forms.

7 Lifecycle Summary

7.1 StatefulExpression lifecycle

```
Parse time:
  parser encounters sum(), avg(), min(), max(), geomean() terminal
    -> creates subclass instance (sum_expression etc.)
    -> constructor captures loop_complex.current_nesting_level()
    -> constructor calls loop_complex.register_stateful(this)
```

```

    -> expression node owns its accumulator state

Run time (driven by LoopComplex::run_once()):
    initialize()  called before first iteration of associated level
    update()      called after every JoSIM run
    finalize()    called when associated level is exhausted
                  (no-op for sum/min/max; meaningful for avg, geomean)

```

7.2 LsqFit lifecycle

```

Parse time:
    parser encounters lsq_fit pragma
        -> creates lsq_fit_function object
        -> captures current_nesting_level
        -> calls loop_complex.register_lsq_fit(this)
        -> stored in separate registry from StatefulExpressions

Run time:
    initialize()  reset data buffer
    add_data()    called after every JoSIM run (feeds the fit)
    finalize()    performs the actual fit computation
                  (always after StatefulExpression::finalize() at same level,
                  making fit results available to expressions defined later in the deck)

```

7.3 Loopiterator lifecycle

```

Parse time:
    parser encounters scan / bsearch pragma
        -> looks up named parameter in symbol table (must already exist)
        -> creates ScanIterator or BinarySearchIterator
        -> constructor calls loop_complex.register_iterator(this)
            loop_complex assigns current_level to iterator
            loop_complex increments current_level
        -> all subsequently parsed StatefulExpressions and LsqFits
            receive this new (higher) level when they self-register
        -> innermost loop = level 0, outermost = highest level

Run time:
    initialize()  set parameter to first value, reset counters
    next()        advance parameter to next value
                  return false when exhausted

```

Note: Statement order in the spice deck determines nesting: the first scan pragma creates the outermost loop (highest level). StatefulExpressions and LsqFits are associated with the innermost loop defined before them, which is the correct level at which to reset and finalize.

8 What Moves Where

Complete migration out of the parameter class:

Currently in parameter	Moves to
scan/bin_search step state	ScanIterator / BinarySearchIterator
iterator[], nesting_level, advance()	LoopComplex
enqueue_zero_op(), zel_element list	StatefulExpression self-registration
enqueue_fit_function(), fit_element list	LsqFit self-registration
enable_sim_anneal flag	main() / Optimizer selection
eval_ptr	LoopComplex
reject_ptr	SimulatedAnnealing (reject_fn)
t0,t1,dt, old/new/min_eval	SA instance members
n_sim_anneal, n_rej, t_max, d_scale	SA instance members
schedule, cur_sched, sched_stp_cntr	SA instance members
sa_log	SA instance member
p_ptr, changed_flag/param/value	SA instance members
save/restore_best, change/unchange_param	SA private methods
have_best, best_val per parameter	SA best-snapshot vector

Stays in parameter: name, cur_value, expr, type, min/max_value, scale, in_summary:1, optimizable:1, last_updated, find_parameter(), get_cur_value(), set_value(), to_normalized(), to_physical(), list_names(), list_values(), update_assignments().

9 Revised main()

```
int main(int argc, char *argv[]) {
    rnd_init(0);
    setup_fifos();
    parse_spice_deck(argv[1]);
    // Parser has built: LoopComplex, parameters,
    //   StatefulExpressions, LsqFits, iterators -- all from pragmas

    LoopComplex loop_complex;

    if (bo_requested) {
        BayesianOptimizer opt(loop_complex);
        opt.configure(bo_config_file);
        opt.run();
    }
}
```

```

    } else if (sa_requested) {
        SimulatedAnnealing opt(loop_complex);
        opt.configure(sa_schedule_file);
        opt.run();
    } else {
        loop_complex.run_once(); // NaN return is fine for parametric sweep
    }

    write_summary_results();
    return 0;
}

```

10 Point 2 -- Bayesian Optimiser Overview

Full implementation detail deferred. Key decisions:

Decision	Choice
Library	BayesOpt or limbo (C++). Choose after prototype evaluation on target platform.
Kernel	Matern 5/2
Acquisition opt	NLopt DIRECT + BOBYQA
Max dimensions	15 (current circuit: 7)
Noisy oracle	GP noise hyperparameter; uncertainty=0.0 at 0K, no API change needed for 4K
Robust centre	Maximise $E[f(x+\delta)]$ on GP surrogate via Monte Carlo
Boundary map	Binary search in ~50 directions from robust centre (~750 oracle calls)
Ellipsoid	MVEE via Eigen; axes = +/- % tolerance per parameter in log space

11 Coding Sequence

Recommended order to minimise regression risk:

- * **Step 1:** Extract StatefulExpression base class and subclasses. Move sum_expressions vector to self-registering pattern. Tests pass.
- * **Step 2:** Extract LoopIterator hierarchy. Move ScanIterator out of parameter. advance() becomes LoopComplex::run_once().
- * **Step 3:** Strip parameter to symbol-table role. Verify ~param~ substitution and summary output unchanged.
- * **Step 4:** Refactor SA as proper class with instance state. Restore original anneal.cpp event-loop structure. run_once() becomes comp_eval().
- * **Step 5:** Add EvalCache between Optimizer and LoopComplex.

- * **Step 6:** Update little language: new keyword tokens in lexer, revised grammar rules, single-statement parameter declarations.
- * **Step 7:** Add BayesianOptimizer stub calling run_once() in budget loop. Wire up chosen library.
- * **Step 8:** Implement robust-centre search, boundary mapping, ellipsoid fitting.

12 Open Questions

- * BayesOpt vs limbo: evaluate build complexity and API ergonomics on target platform before committing.
- * BinarySearchIterator probe expression: does it reference an existing named parameter expression, or does it need a new pragma keyword? The current grammar uses SYMBOL references for all such expressions.
- * Flex/Bison: renaming ASSIGN to EQUALS (Point 3 cleanup) should be done atomically across lexer and grammar to avoid a broken intermediate state.
- * 4K thermal noise: oracle should anticipate returning (value, uncertainty) pair even while uncertainty=0.0 at 0K, so no API change is needed later.
- * LoopComplex ownership: since it is built by the parser via global state (Flex/Bison C API), clarify how the constructed LoopComplex object is handed to main() -- likely via a global or a parser_interf.h accessor function, consistent with existing pattern.

End of document